The Humanized Guide to EcoBuck's Backend & Connectivity

Making Sense of the Cloud, Database, and Telemetry

Hey there! If you're a developer, product manager, or designer working on EcoBuck, this guide is for you. We've stripped away the dense academic jargon and architecture diagrams to explain **how the EcoBuck backend actually works**, **why we built it this way**, and **how the device, the cloud, and your phone talk to each other**.

---

1. What is EcoBuck and Why Do We Need a Backend?

EcoBuck is a smart compost bin. It has an ESP32 microchip inside, connected to temperature and moisture sensors.

But a smart bin isn't very smart if it can't tell you how it's doing. * **The ESP32 firmware (built by Team B)** does the heavy lifting inside the bin. It collects raw sensor readings, runs math to clean up noise (like sudden temperature spikes if someone drops a warm cup of tea near it), and tracks whether the compost is actively heating up, cooling down, or ready. * **The Backend (that's us!)** is the bridge. Since the bin has limited memory and goes to sleep to save battery, it can't run a web server or talk directly to your phone. It sends its data to the cloud (the backend). * **The Mobile App (Team D)** talks to the backend to show you a nice, clean dashboard with status indicators, history graphs, and push notifications.

---

2. The Core Workflow (How Data Travels)

Think of data moving through EcoBuck in a simple loop:

1. **Read & Clean:** The bin takes a sensor reading and filters it.
2. **Report:** The bin connects to Wi-Fi and says, *"Hey backend, here is my latest update!"* in a structured JSON payload.
3. **Store:** The backend writes this update into the Firestore database.
4. **Check Rules:** The backend immediately runs a quick check on the data. Is it too wet? Is the device suddenly offline? Is the compost ready?

5. **Alert:** If anything is wrong (or if the compost is ready!), the backend sends a push notification through Firebase Cloud Messaging (FCM) straight to the user's phone.
6. **Display:** When the user opens the EcoBuck app, it displays the current status and historical graphs.

---

3. Our Key Database Secret: Caching the Latest Status

If you've ever used Firestore, you know that Google charges you for every single document you read. If the app had to read the last 500 telemetry logs every single time you opened the dashboard just to find the current temperature, your cloud bill would skyrocket, and the app would feel sluggish.

To solve this, we use a pattern called **Write-Duplication**: * Whenever the device sends an update, the backend writes it in two places: 1. A long-term log of all readings (under `/telemetry` subcollection) for charts. 2. An overwrite of a single document called `/latest/status`. * When the user opens the mobile dashboard, the app only reads `/latest/status`. It gets the current state instantly, costs only **1 read charge**, and loads in milliseconds!

---

4. The Rules & Alert Engine (Our System's Brain)

The backend has rules running automatically to watch over the composting process. Here's what it alerts you about: * **"Is the bin alive?" (Offline Alert):** The bin is supposed to report periodically. If we haven't heard from it in a while, the backend flags it as offline. * **"Is a sensor broken?" (Sensor Error Alert):** If the ESP32 reports that a probe has disconnected or is returning garbage values, the backend alerts the user to check the hardware. * **"Is it too dry or too wet?":** Microbes need moisture to break down organic waste. If moisture drops below 20%, composting stops. If it goes above 75%, it becomes anaerobic and starts to smell terrible. The backend warns the user to add water or dry waste (like cardboard). * **"It's Ready!":** When conditions remain stable near room temperature for 2.5 days after at least two weeks of composting, the backend triggers a "Ready Candidate" alert.

---

5. Manual Validation (The Handshake)

We don't want to rely 100% on sensors to claim compost is ready. Sensors are smart, but your eyes and nose are smarter. 1. When the backend decides the compost is ready, it marks the status as `ready_candidate`. 2. The app shows the user a prompt: *"Your compost looks ready! Can you confirm it is dark brown, crumbly, and smells like forest soil?"* 3. When the user taps "Yes, confirm", the app calls our `/validate` API. 4. The backend changes the state to `ready_confirmed`. On the next sync cycle, the device receives this update, updates its local status, and resets the composting cycle clock for your next

batch of waste!

---

6. Cool Future Stuff (AI Integrations)

Once we have enough historical data from multiple composting bins, we can plug in cool AI models: * **Compost Predictor:** Instead of guessing, we can analyze the temperature curve and say: *"Your compost is 68% complete, estimated ready on Friday!"* * **Odor Prevention:** We can detect anaerobic patterns before they smell, warning you: *"Hey, your pile is compacting. Turn it now to prevent bad odors!"* * **Ambient Adaptation:** If a bin is kept on a freezing balcony in winter vs. a hot kitchen in summer, the AI adjusts the threshold math automatically so you don't get false alarms.